Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07

ПРАКТИЧЕСКАЯ РАБОТА № 3

Дисциплина МДК 03.02

Аудит кода на соответствие документам национальной системы

Вариант № 2
АИС «Умный кампус»
Модуль Library System

Студент: Шваб Роберт Эдуардович
Группа: И13_24_11
Подпись: _____

Руководитель: _____
Дата представления: «___» _____ 2026 г.

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

1 ВЫПОЛНЕНИЕ РАБОТЫ.....3

1.1 Методика и границы аудита.....3

1.2 Исправление несоответствий.....4

1.3 Результат аудита.....4

1.4 Выводы.....6

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....8

ПРИЛОЖЕНИЕ А Листинг исходного кода.....9

1 ВЫПОЛНЕНИЕ РАБОТЫ

Я выполнил работу по варианту 2 Library System АИС «Умный кампус» на C# и .NET 8. Модуль принимает PDF/EPUB/ZIP в карантин; права Reader и Librarian разделены [3]. Оформление выполнено по положению института [2].

1.1 Методика и границы аудита

Я сравнил LegacyUploadController.cs с рабочими UploadController.cs и BooksController.cs. Правила PascalCase, имён полей, XML-комментариев и структуры обязанностей заданы политикой проекта и .editorconfig. Соглашения Microsoft приняты как основа именования. Эти правила не приписываются ГОСТ 19.201-78 [4].

Отдельно я проверил ТЗ на восемь обязательных разделов по ГОСТ 19.201-78: введение; основания; назначение; требования к программе; документация; технико-экономические показатели; стадии; контроль и приёмка. Все разделы присутствуют. Я проверил связь назначения, FR и критериев вручную: наличие заголовка не доказывает содержательную полноту [1].

Сведения представлены в таблице 1.

Таблица 1 — Протокол исходного контроллера

Правило	Объект	Статус	Замечание
AUD-01	upload	Не соответствует	Имя метода не PascalCase
AUD-02	upload	Требует доработки	Нет XML summary
AUD-01	count	Не соответствует	Имя метода не PascalCase
AUD-02	count	Требует доработки	Нет XML summary
AUD-04	ZIP	Не соответствует	Непроверенный путь
AUD-05	role	Не соответствует	Reader допущен к записи
AUD-06	Root	Не соответствует	Приватное поле не _camelCase
AUD-07	Controller	Не соответствует	Нет серверной политики

Источник: составлено автором по результатам работы

1.2 Исправление несоответствий

Основные логические дефекты — запись по недоверенному имени ZIP, допуск Reader к записи и сохранение произвольных байтов. Исходный пример исследован статически и не включён в маршруты API. В рабочем решении авторизацию выполняет сервер, формат проверяет FilePolicy, сохранение — UploadService.

Сведения представлены в таблице 2.

Таблица 2 — Чек лист после исправления

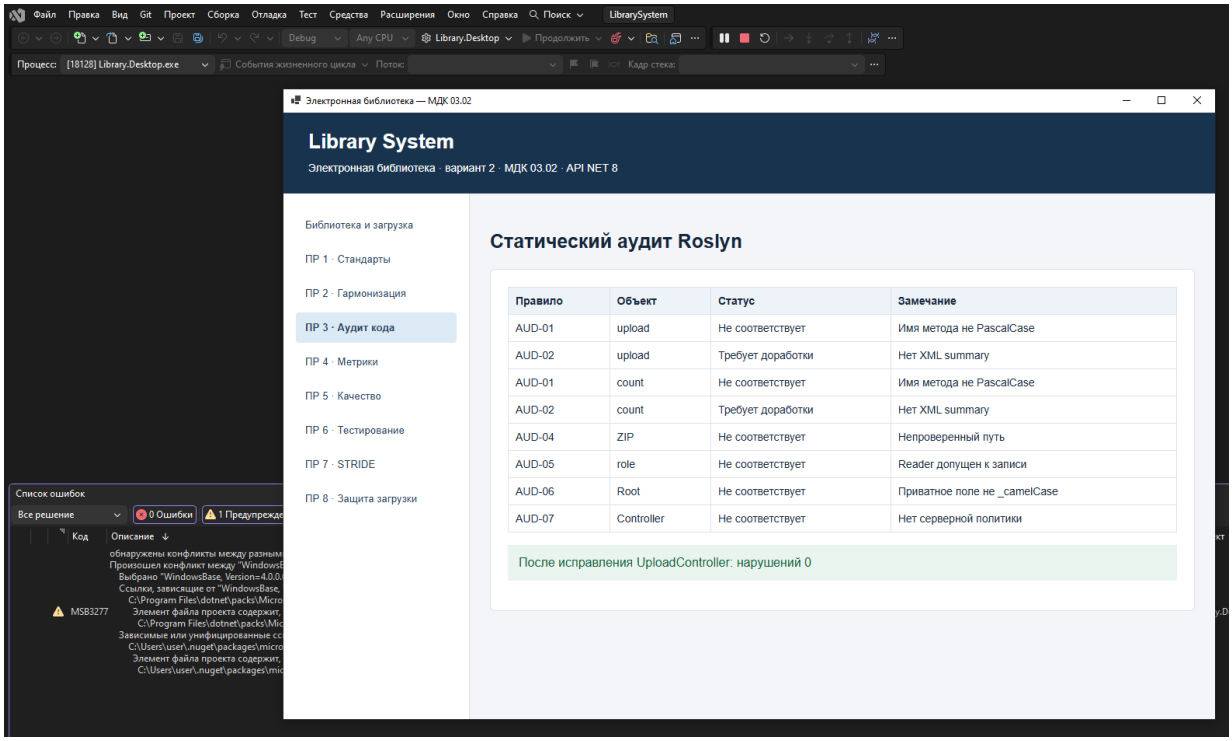
Проверка	Статус	Основание
Публичный метод PascalCase	Соответствует	Upload
XML summary	Соответствует	Описание класса и метода
Одна обязанность контроллера	Соответствует	Сохранение вынесено в сервис
Безопасные ZIP-пути	Соответствует	Проверка и тесты до записи
Структура ТЗ	Соответствует	Восемь разделов LS-TZ-01

Источник: составлено автором по результатам работы

1.3 Результат аудита

По заданным правилам Roslyn найдено 8 записей в исходном контроллере, 0 — в UploadController и 0 — в BooksController. Это ограниченный набор проверок, а не утверждение о полном SAST всего решения. Статусы «Не соответствует» и «Требует доработки» сохранены в протоколе; «Соответствует» показан для проверенного исправления.

На рисунке 1 показаны результаты восьми замечаний к исходному контроллеру и итог проверки исправленного UploadController.



Источник: снимок прогона на Windows, предоставленный автором.

Рисунок 1 — Результаты статического аудита исходного контроллера по правилам Roslyn

Я запустил Library.Tools из Visual Studio. В консольном протоколе на рисунке 2 сохранены восемь исходных замечаний; массивы afterAudit и catalogAudit пусты. Процесс завершился с кодом 0. Это результаты правил проекта на Roslyn, а не встроенного анализатора Visual Studio.

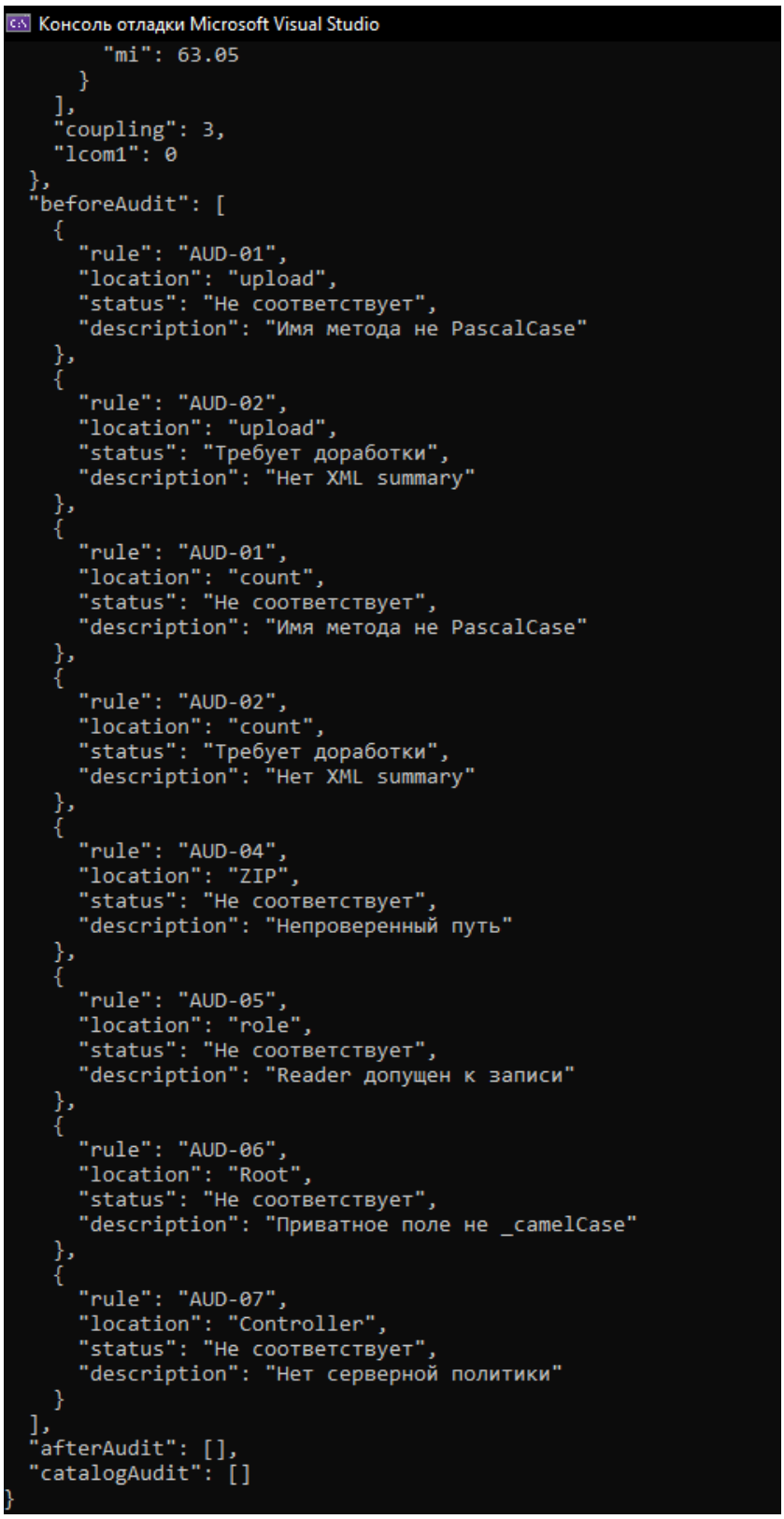


Рисунок 2 — Консольный протокол аудита Library.Tools в Visual Studio

1.4 Выводы

Я разделил замечания к именованию, документации и логике и проверил результат исправления. ГОСТ использован по назначению — для

ТЗ; стиль C# контролируется отдельной политикой. Наиболее существенное изменение — устранение доверия к пути и клиентскому назначению роли.

Полные исходники, входные данные и журналы переданы с отчётом. Основной листинг приведён в приложении А.

Исходный код и порядок запуска: репозиторий MDK0302-LibrarySystem.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19.201-78. ЕСПД. Техническое задание. Требования к содержанию и оформлению. — URL: https://protect.gost.ru/gost/details/b64061cf-6bb2-4c75-aee9-18bb5e26e554 (дата обращения: 08.11.2026) [1].

2. Положение об оформлении письменных работ. Санкт-Петербургский институт экономики и управления. Методические материалы [2].

3. Практические работы МДК 03.02. Задания ПР 1–8 и эссе. Вариант 2 Library System. Материалы преподавателя [3].

4. Microsoft. C# identifier naming rules and conventions. — URL: https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/identifier-names (дата обращения: 08.11.2026) [4].

ПРИЛОЖЕНИЕ А
Листинг исходного кода

before/LegacyUploadController.cs

```
using System.IO.Compression;
namespace Library.Before;
// Изолированный исходный образец. Не включается в сборку API.
public class LegacyUploadController
{
   private string Root = "App_Data";
   public string upload(string role, string name, byte[] bytes)
   {
      if (role == "Librarian" || role == "Reader")
      {
         if (bytes.Length == 0) return "error";
         if (name.EndsWith(".zip"))
         {
            using var zip = new ZipArchive(new MemoryStream(bytes));
            foreach (var entry in zip.Entries)
            {
               string path = Path.Combine(Root, entry.FullName);
               Directory.CreateDirectory(Path.GetDirectoryName(path)!);
               using var input = entry.Open();
               using var output = File.Create(path);
               input.CopyTo(output);
            }
         }
         else File.WriteAllBytes(Path.Combine(Root, name), bytes);
         return "ok";
      }
      return "error";
   }
   public int count() => Directory.GetFiles(Root).Length;
}
```

src/Library.Api/UploadController.cs

```
using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.RateLimiting;
namespace Library.Api;


/// <summary>Принимает книги только от библиотекаря.</summary>
[ApiController]
[Route("api/books")]
[Authorize(Policy = "LibrarianOnly")]
public sealed class UploadController(UploadService service) : ControllerBase
{
   /// <summary>Создаёт записи карантина и возвращает 201.</summary>
   [HttpPost("upload")]
   [EnableRateLimiting("uploads")]
   [RequestSizeLimit(2 * 1024 * 1024 + 65536)]
   public async Task<IActionResult> Upload([FromForm] string title, IFormFile file, CancellationToken ct)
   {
      string actor = User.FindFirstValue(ClaimTypes.NameIdentifier)!;
```

```
        var rows = await service.SaveAsync(title, file, actor, ct);
        return StatusCode(201, rows.Select(x => new { x.Id, x.Title, x.OriginalName,
            x.Length, x.Sha256, Status = "Quarantined" }));
    }
}
```

src/Library.Tools/CodeQuality.cs

```
using System.Text.RegularExpressions;
using Microsoft.CodeAnalysis;
using Microsoft.CodeAnalysis.CSharp;
using Microsoft.CodeAnalysis.CSharp.Syntax;
namespace Library.Tools;
public sealed record MethodMetric(string Name, int Lines, int Complexity, double Volume, double Mi);
public sealed record ClassMetric(string Name, MethodMetric[] Methods, int Coupling, int Lcom1);
public sealed record Finding(string Rule, string Location, string Status, string Description);
/// <summary>Воспроизводимый синтаксический анализ Roslyn; не семантический CBO.</summary>
public static class CodeQuality
{
    public static ClassMetric Measure(string source, string name)
    {
        var root = CSharpSyntaxTree.ParseText(source).GetCompilationUnitRoot();
        var type = root.DescendantNodes().OfType<ClassDeclarationSyntax>().Single(x => x.Identifier.ValueText
== name);
        var methods = type.Members.OfType<MethodDeclarationSyntax>().ToArray();
        var ignored = new HashSet<string> { "Task", "List", "Stream", "Guid", "CancellationToken", "Exception" };
        int coupling = type.DescendantNodes().OfType<TypeSyntax>()
            .Where(x => x.Parent is ParameterSyntax or VariableDeclarationSyntax or
ObjectCreationExpressionSyntax or BaseTypeSyntax)
            .Select(x => x.ToString()).Where(x => x.Length > 0 && char.IsUpper(x[0]) && !ignored.Contains(x) &&
x != name).Distinct().Count();
        var fields = type.Members.OfType<FieldDeclarationSyntax>().SelectMany(x => x.Declaration.Variables)
            .Select(x => x.Identifier.ValueText).ToHashSet();
        if (type.ParameterList is not null)
            fields.UnionWith(type.ParameterList.Parameters.Select(x => x.Identifier.ValueText));
        var access = methods.Select(x => x.DescendantNodes().OfType<IdentifierNameSyntax>()
            .Select(y => y.Identifier.ValueText).Where(fields.Contains).ToHashSet()).ToArray();
        int shared = 0, separate = 0;
        for (int i = 0; i < access.Length; i++) for (int j = i + 1; j < access.Length; j++)
            if (access[i].Overlaps(access[j])) shared++; else separate++;
        return new(name, methods.Select(Method).ToArray(), coupling, Math.Max(0, separate - shared));
    }
    public static MethodMetric Method(MethodDeclarationSyntax method)
    {
        int cc = 1;
        foreach (var node in method.DescendantNodes())
        {
            if (node is IfStatementSyntax or ForStatementSyntax or ForEachStatementSyntax or
                WhileStatementSyntax or DoStatementSyntax or CatchClauseSyntax or
                ConditionalExpressionSyntax or CaseSwitchLabelSyntax) cc++;
            if (node is BinaryExpressionSyntax b && (b.IsKind(SyntaxKind.LogicalAndExpression) ||
                b.IsKind(SyntaxKind.LogicalOrExpression) || b.IsKind(SyntaxKind.CoalesceExpression))) cc++;
        }
        SyntaxNode body = (SyntaxNode?)method.Body ?? (SyntaxNode?)method.ExpressionBody ?? method;
        int lines = body.ToString().Split('\n').Count(x => x.Trim().Length > 0 && !x.TrimStart().StartsWith("//"));
        var tokens = body.DescendantTokens().ToArray();
        bool Operand(SyntaxToken t) => t.IsKind(SyntaxKind.IdentifierToken) ||
t.IsKind(SyntaxKind.StringLiteralToken) || t.IsKind(SyntaxKind.NumericLiteralToken);
        int n = tokens.Where(Operand).Select(x => x.Text).Distinct().Count() + tokens.Where(x => !
Operand(x)).Select(x => x.Text).Distinct().Count();
        double volume = tokens.Length * Math.Log2(Math.Max(2, n));
```

```
      double mi = Math.Clamp((171 - 5.2 * Math.Log(Math.Max(1, volume)) - .23 * cc - 16.2 *
Math.Log(Math.Max(1, lines))) * 100 / 171, 0, 100);
      return new(method.Identifier.ValueText, lines, cc, Math.Round(volume, 2), Math.Round(mi, 2));
    }
    public static Finding[] Audit(string source)
    {
      var root = CSharpSyntaxTree.ParseText(source).GetCompilationUnitRoot();var list = new List<Finding>();
      foreach (var m in root.DescendantNodes().OfType<MethodDeclarationSyntax>().Where(x =>
x.Modifiers.Any(SyntaxKind.PublicKeyword)))
      {
        string name = m.Identifier.ValueText;
        if (!Regex.IsMatch(name, "^[A-Z][A-Za-z0-9]*$")) list.Add(new("AUD-01", name, "Не соответствует",
"Имя метода не PascalCase"));
        if (!m.GetLeadingTrivia().ToFullString().Contains("<summary>")) list.Add(new("AUD-02", name,
"Требует доработки", "Нет XML summary"));
        if (Method(m).Complexity > 10 || Method(m).Lines > 30) list.Add(new("AUD-03", name, "Требует
доработки", "Сложный или длинный метод"));
      }
      if (source.Contains("Path.Combine(Root, entry.FullName)")) list.Add(new("AUD-04", "ZIP", "Не
соответствует", "Непроверенный путь"));
      if (source.Contains("role == \"Reader\"")) list.Add(new("AUD-05", "role", "Не соответствует", "Reader
допущен к записи"));
      if (source.Contains("private string Root")) list.Add(new("AUD-06", "Root", "Не соответствует",
"Приватное поле не _camelCase"));
      if (!source.Contains("Authorize")) list.Add(new("AUD-07", "Controller", "Не соответствует", "Нет
серверной политики"));
      return list.ToArray();
    }
}
```

src/Library.Api/BooksController.cs

```
using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
namespace Library.Api;

/// <summary>Читает каталог с параметризованным LINQ и изоляцией карантина.</summary>
[ApiController]
[Route("api/books")]
[Authorize]
public sealed class BooksController(LibraryDb db, StorageRoot root) : ControllerBase
{
  /// <summary>Не раскрывает чужие неопубликованные документы.</summary>
  [HttpGet]
  public async Task<IActionResult> Get([FromQuery] string? query, CancellationToken ct)
  {
    string actor = User.FindFirstValue(ClaimTypes.NameIdentifier)!;
    bool librarian = User.IsInRole("Librarian");
    var books = db.Books.Where(x => x.Published || (librarian && x.Owner == actor));
    if (!string.IsNullOrWhiteSpace(query)) books = books.Where(x => x.Title.Contains(query));
    return Ok(await books.Select(x => new { x.Id, x.Title, x.OriginalName, x.Length,
      Status = x.Published ? "Published" : "Quarantined" }).ToArrayAsync(ct));
  }
  /// <summary>Выдаёт только опубликованный образец вне веб-каталога.</summary>
  [HttpGet("{id:guid}/file")]
  public async Task<IActionResult> Download(Guid id, CancellationToken ct)
  {
    var book = await db.Books.SingleOrDefaultAsync(x => x.Id == id && x.Published, ct);
```

```
        if (book is null) return NotFound();
        Response.Headers["X-Content-Type-Options"] = "nosniff";
        return PhysicalFile(Path.Combine(root.Path, "published", book.StoredName),
            "application/octet-stream", book.OriginalName);
    }
}
```